Challenges

Our main challenge during this project was designing the overall class structure for each module. We initially decided on having a Problem class to represent each Problem state, a Node class to wrap the Problem classes, and a Tree class to traverse nodes. However, when implementing the A* search algorithm, we faced difficulties understanding how to traverse the Node classes with the Problem class wrapped inside. To fix this, we removed the necessary functions outside the Problem and Node classes, and implemented the A* search in one file. Then, we split the methods into separate files to prevent the A* algorithm file from becoming too cluttered.

Design

The only object we used in our final version was a Node class, which has attributes f, g, grid, and parent. The rest of our project is split into methods for heuristics, helpers, and the actual A* algorithm. We have two methods for the heuristics (**heuristic_misplaced_tile** and **heuristic_euclidean_distance**), four methods for helpers (**find_blank, apply_swap, generate_goal,** and **grid_to_tuple**), along with an enumeration class named **Swap** that holds tuples for the **UP, DOWN, LEFT,** and **RIGHT** directions. We also have an **InvalidMoveError** exception class in the helpers to prevent invalid states from being generated. Lastly, our A-star file holds the **A_star** method, and the main file contains the driver code.

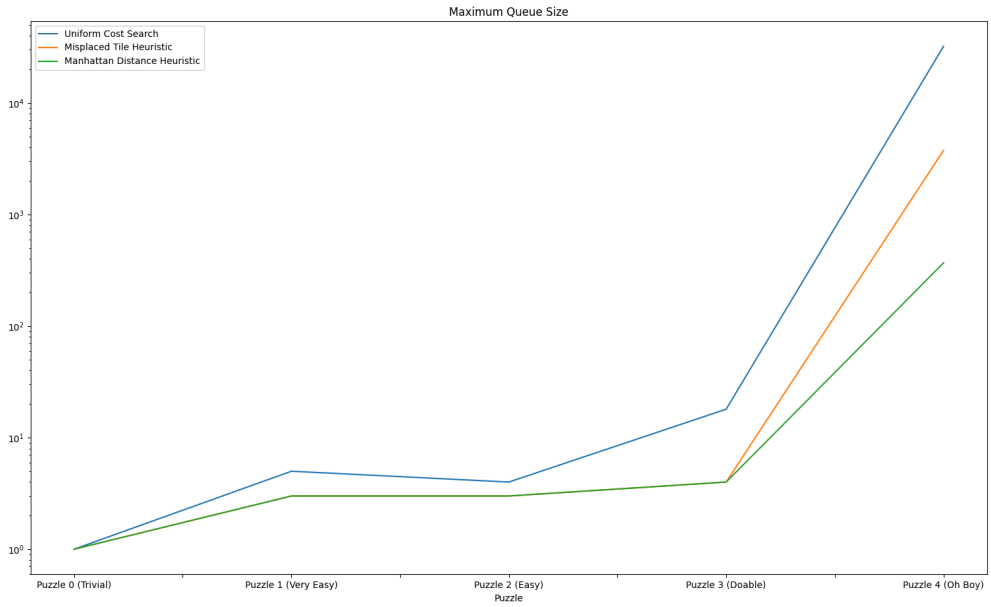
Optimization

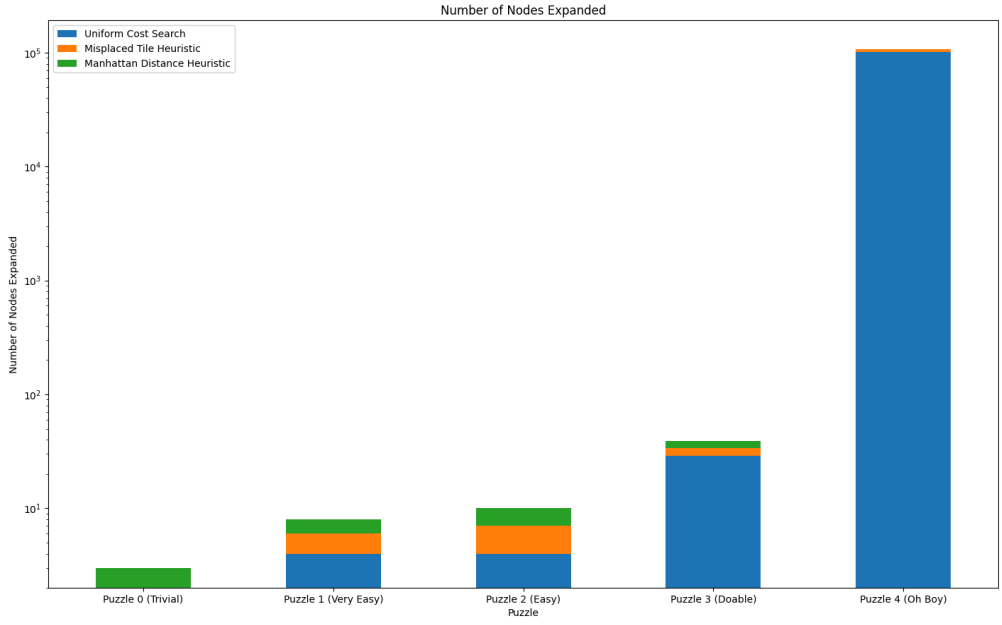
We use a min-heap for the frontier, and for the visited nodes, we use a set. With a heap, the state with the smallest heuristic is stored at the top of the heap. So, instead of searching through the heap in O(n) time, we can pop off the smallest Node in O(logn) time. We can look up the visited nodes in a set in O(1) time instead of O(n) since Python implements sets as hash tables. This operation makes the search significantly faster, especially as the frontier and explored sets grow.

Tree or Graph Search?

We implemented a graph search in our project by keeping track of the explored Nodes in a set outside of the search loop.

Analysis





We observed that more challenging puzzles require more node expansions, which takes longer to find the goal state. There is also a noticeable difference in how large the queue size becomes for different heuristic functions when completing more challenging puzzles. The Manhattan Distance heuristic outperformed all other heuristic functions for increasingly challenging puzzle states.